System Architecture and Repository Foundations

Overview

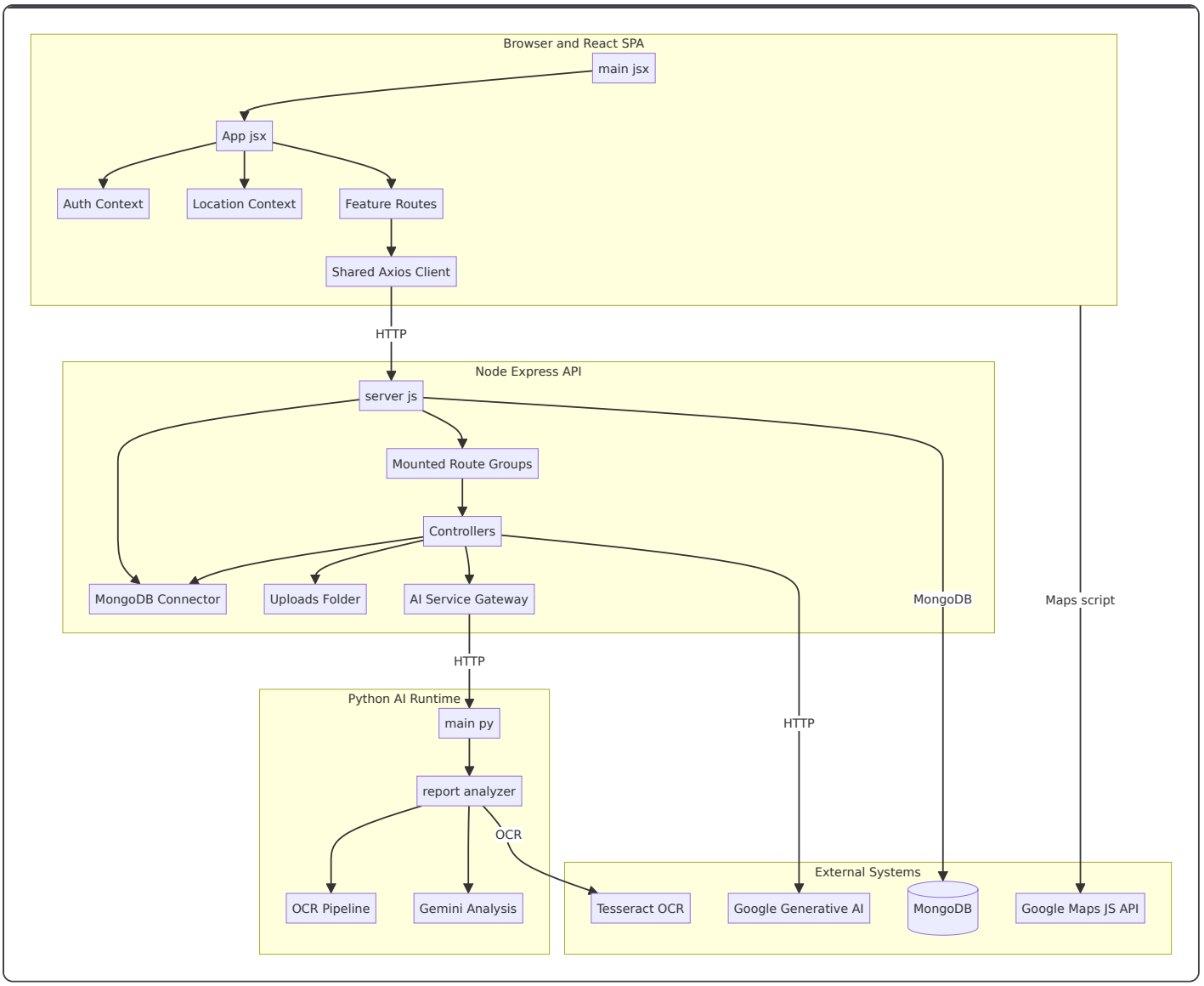
MedAccess is organized as a three-runtime monorepo: a Node.js/Express backend, a React 18 + Vite browser SPA, and an optional FastAPI-based Python AI service. The browser experience starts in `frontend/src/main.jsx`, flows through `frontend/src/App.jsx`, and then fans out into feature pages that call the shared Axios client in `frontend/src/services/api.js`. The backend starts in `backend/server.js`, mounts the API surface, connects to MongoDB through `backend/config/db.js`, serves uploads, and brokers AI requests to the Python service when needed.

The repository began around location-based healthcare discovery and now spans hospitals, pharmacies, blood donation, blood requests, resource reporting, appointments, biometrics, hospital administration, and AI-assisted medical analysis. The expansion is visible in the mounted route groups on the Express side and in the SPA route set in `frontend/src/App.jsx`, where the original hospital locator flows now sit beside blood workflows, reporting tools, admin portals, and the AI assistant.

Monorepo Topology

Area	Entry Points	Responsibility	Runtime Boundary
`backend/`	`server.js`, `config/db.js`, route modules, controllers, models	Express API, MongoDB persistence, upload handling, AI gateway, protected workflows	Node.js process
`frontend/`	`src/main.jsx`, `src/App.jsx`, `src/services/api.js`, `src/context/*.jsx`, `src/pages/*.jsx`	React SPA shell, route layout, session state, location state, API consumption	Browser runtime
`ai-service/`	`main.py`, `report_analyzer.py`	FastAPI AI runtime, OCR, Gemini-based report analysis, ML support files	Python process
Repository root	Operational launch and deployment assets referenced by the project structure	Process orchestration across the three runtimes	Host and deployment boundary

Architecture Overview



Runtime Boundaries and Service Composition

Node.js and Express boundary

> **NOTE**
> The default browser API base URL in `frontend/src/services/api.js` is `http://localhost:5001/api`, while the Express server in `backend/server.js` listens on `process.env.PORT || 5000`. When no environment variables override those values, the SPA points at a different port than the backend process.

`backend/server.js`, `backend/config/db.js`, `backend/controllers/*.js`, `backend/routes/*.js`

The Express runtime owns authenticated API flows, persistence, file upload handling, and proxy calls to the AI service. `backend/server.js` connects middleware in a fixed order: CORS, JSON parsing, URL-encoded parsing, request logging, route mounting, static upload serving, a health endpoint, a 404 handler, and the error handler.

Browser and React boundary

`frontend/src/main.jsx`, `frontend/src/App.jsx`, `frontend/src/context/*.jsx`, `frontend/src/services/*.js`

The SPA boundary bootstraps once in `frontend/src/main.jsx`, then `frontend/src/App.jsx` wraps routing in `AuthProvider` and `LocationProvider`. The UI shell conditionally hides the shared navbar, sidebar, and footer for full-screen authentication and admin routes, while feature pages reuse the same HTTP client and context state.

Python AI boundary

`ai-service/main.py`, `ai-service/report_analyzer.py`

The optional AI runtime is isolated in Python. `backend/controllers/aiController.js` sends symptom and report-analysis requests to this service, which is prepared for OCR-driven report interpretation and Gemini-backed response generation.

External service boundary

Browser Google Maps script, MongoDB, Gemini, Tesseract

The browser loads Google Maps JS directly in `frontend/index.html`. The backend uses MongoDB through Mongoose, the AI service uses Tesseract and Gemini, and the Express AI gateway forwards requests to the Python runtime via `AI_SERVICE_URL`.

Frontend Boot and Shell

`frontend/src/main.jsx`

> **NOTE**
> The route `POST /api/reports/:id/analyze` is wired through `backend/controllers/aiController.js`, but that handler returns the stored `Report` with a `"Feature coming soon"` message instead of invoking the Python OCR/Gemini pipeline. The endpoint is reachable, but it does not perform analysis yet.

`frontend/src/main.jsx`

This is the browser entry point. It imports `App`, loads the global stylesheet, and mounts the React tree into `#root` with `ReactDOM.createRoot`.

`frontend/src/App.jsx`

`frontend/src/App.jsx`

`App.jsx` defines the SPA shell and route surface. It creates the router, wires `AuthProvider` and `LocationProvider`, and uses `AppContent` to manage layout visibility and route transitions.

Property	Type	Description
`sidebarOpen`	`boolean`	Tracks whether the shared sidebar is open
`location`	`Location` from `react-router-dom`	Current route location used for layout decisions
`noLayoutPaths`	`string[]`	Routes that render without the shared navbar, sidebar, or footer
`showLayout`	`boolean`	Controls whether shared chrome is rendered

Route shell behavior

- Routes under `/login`, `/register`, `/register-hospital`, `/hospital-admin`, `/admin`, `/pharmacy-login`, `/pharmacy-register`, and `/pharmacy-admin` render without the shared layout.
- Route changes close the sidebar automatically.
- `ToastContainer` is always present at the shell level.

State Management and Session Flow

frontend/src/context/ AuthContext.jsx

`frontend/src/context/AuthContext.jsx`

`AuthContext` persists authenticated user state in `localStorage`, exposes login/logout/update helpers, and provides the `isAuthenticated` flag used by route-level and component-level guards.

Property	Type	Description	
`user`	`object \	null`	Current authenticated user object
`token`	`string \	null`	JWT token loaded from `localStorage`
`loading`	`boolean`	Shared loading flag for auth-driven operations	
`setLoading`	`function`	Updates the loading flag	
`login`	`function`	Stores user data and token in context and `localStorage`	
`logout`	`function`	Clears session state and removes auth items from `localStorage`	
`updateUser`	`function`	Merges profile updates into the current user object	
`isAuthenticated`	`boolean`	Derived from the presence of a token	

frontend/src/context/ LocationContext.jsx

`frontend/src/context/LocationContext.jsx`

`LocationContext` manages browser geolocation for discovery, emergency, and resource workflows.

Property	Type	Description			
`location`	`{ latitude: number \	null, longitude: number \	null, address: string, loading: boolean, error: string \	null }`	Current locatic state
`getLocation`	`function`	Requests browser geolocation and updates `location`			
`setCustomLocation`	`function`	Writes a manual location into the shared state			

Location state transitions

- `loading: true` while geolocation is in progress.
- `loading: false, error: null` after a successful lookup.
- `loading: false, error: <message>` when geolocation fails or is unavailable.

Shared API Client and HTTP Facades

Axios API Client

`frontend/src/services/api.js`

This file is the shared transport layer for the SPA. All feature facades import the same Axios instance, so auth headers, base URL selection, and 401 handling are centralized.

Property	Type	Description
`api`	`AxiosInstance`	Shared HTTP client exported as the default module value
`API_BASE_URL`	`string`	Uses `import.meta.env.VITE_API_URL` or falls back to `http://localhost:5001/api`

Behavior	Description
Request headers	Sets `Content-Type: application/json` by default
Request interceptor	Reads `token` from `localStorage` and adds `Authorization: Bearer <token>` when present
Debug logging	Logs base URL, outgoing requests, and response errors to the console
401 handling	Clears `token` and `user` from `localStorage` and redirects to `/login`

Frontend API facades

`frontend/src/services/authService.js`

This file aggregates the feature-specific HTTP facades used throughout the SPA.

`authService`

Method	Description
`register`	Creates a standard user account
`registerHospital`	Creates a hospital administrator account
`login`	Authenticates a user and returns a session token
`getProfile`	Fetches the current user profile
`updateProfile`	Updates the current user profile

`hospitalService`

Method	Description
`getNearbyHospitals`	Fetches nearby hospitals for a latitude, longitude, and radius
`getHospitalDetails`	Fetches detailed information for a place ID

`reportService`

Method	Description
`uploadReport`	Uploads a medical report file
`getMyReports`	Lists the current user's reports
`deleteReport`	Deletes a report by ID
`analyzeReport`	Requests analysis for an existing report by ID

`resourceService`

Method	Description
`reportResource`	Submits a medical resource report
`getVerifiedResources`	Lists verified resources, optionally filtered
`getPendingResources`	Lists pending resources for review
`getMyReports`	Lists the caller's resource reports
`verifyResource`	Marks a resource as verified
`rejectResource`	Marks a resource as rejected
`deleteResource`	Deletes a resource report

`biometricService`

Method	Description
`record`	Stores a biometric reading
`getHistory`	Fetches the caller's biometric history
`getLatest`	Fetches the latest biometric snapshot

appointmentService

Method	Description
`book`	Books a new appointment
`getMine`	Lists the caller's appointments
`update`	Updates an appointment by ID
`cancel`	Cancels an appointment by ID

pharmacyService

Method	Description
`getNearbyPharmacies`	Fetches nearby pharmacies
`getNearbyEmergencyServices`	Fetches nearby emergency services surfaced by the pharmacy API

bloodRequestService

Method	Description
`createRequest`	Creates a blood request
`getRequests`	Lists blood requests with optional filters
`getMyRequests`	Lists the caller's blood requests
`updateRequest`	Updates a blood request by ID
`deleteRequest`	Deletes a blood request by ID

`ambulanceService`

Method	Description
`registerAmbulance`	Registers an ambulance
`updateLocation`	Updates ambulance coordinates
`updateStatus`	Updates ambulance status
`getNearbyAmbulances`	Lists nearby ambulances
`trackAmbulance`	Tracks one ambulance by ID
`callAmbulance`	Sends a call request to an ambulance
`getAllAmbulances`	Lists all ambulances

`adminService`

Method	Description
`getDashboard`	Fetches hospital dashboard statistics
`getAppointments`	Fetches hospital appointments with optional filters
`updateAppointment`	Updates appointment status and notes
`getPatients`	Fetches hospital patient records
`updateProfile`	Updates the hospital profile

Backend Runtime

backend/ `server.js`

`backend/server.js`

`server.js` is the Express runtime entry point. It initializes the app, calls `connectDB()`, installs middleware, mounts route groups, exposes uploads, and starts the HTTP listener.

Step	Behavior	
1	Creates the Express app	
2	Connects to MongoDB through `connectDB()`	
3	Enables CORS with development and production origin rules	
4	Parses JSON and URL-encoded payloads	
5	Logs each request with timestamp, method, and path	
6	Mounts route groups under `/api/...`	
7	Serves `backend/uploads` under `/uploads`	
8	Exposes `/health`	
9	Returns JSON for unmatched routes	
10	Delegates final failures to `errorHandler`	
11	Listens on `process.env.PORT	5000`

backend/config/ db.js

`backend/config/db.js`

This module owns the MongoDB connection lifecycle.

Method	Description
`connectDB`	Connects to `process.env.MONGODB_URI`, logs the host on success, and exits the process on failure

backend/controllers/ aiController.js

`backend/controllers/aiController.js`

This module is the Node-side gateway to the optional Python AI runtime.

Method	Description
`diagnoseSymptoms`	Validates a symptom array, calls the AI service `/predict`, stores a `Diagnosis`, and returns the AI output
`analyzeReportFile`	Accepts an uploaded file, forwards it to the Python AI service, deletes the temp file, and returns the AI output
`analyzeReport`	Returns a stored `Report` record for an existing report ID and marks the feature as coming soon

AI gateway behavior

- Uses `AI_SERVICE_URL` from the environment or `http://localhost:8000`.
- Sends symptom payloads as JSON.
- Sends report files as multipart form-data.
- Cleans up temporary uploaded files with `fs.unlinkSync`.
- Returns upstream AI errors and a service-unavailable response when the Python runtime is unreachable.

backend/controllers/ bloodRequestController.js

`backend/controllers/bloodRequestController.js`

Method	Description
`createBloodRequest`	Validates required fields, stores a blood request, and returns a compact request summary
`getBloodRequests`	Filters pending blood requests by blood group, city, urgency, and optional geospatial radius
`getMyBloodRequests`	Returns the caller's blood requests
`updateBloodRequest`	Updates status and notes for a blood request
`deleteBloodRequest`	Deletes a blood request

backend/controllers/ resourceController.js

`backend/controllers/resourceController.js`

Method	Description
`reportResource`	Validates resource fields, stores a resource report, and returns a compact report summary
`getVerifiedResources`	Filters verified resources by type, city, and optional geospatial radius
`getPendingResources`	Returns pending resource reports for admin review
`verifyResource`	Marks a resource verified and records the verifying user
`rejectResource`	Marks a resource rejected
`getMyReports`	Returns the caller's resource reports
`deleteResource`	Deletes a resource report

Backend request logging

The Express runtime prints every request with an ISO timestamp, method, and path. The AI gateway also logs upstream failures and service availability errors. This gives the repository a lightweight, code-level observability layer without a separate telemetry service.

AI Runtime

`ai-service/main.py`

`ai-service/main.py`

`main.py` creates the FastAPI app that serves the optional AI runtime.

Property	Type	Description	
`app`	`FastAPI`	FastAPI application titled `MediAccess AI Service`	
`MODEL_PATH`	`string`	Model file path set to `model.pkl`	
`SYMPTOMS_PATH`	`string`	Symptoms file path set to `symptoms.pkl`	
`model`	`object \	None`	Loaded prediction model placeholder
`symptoms_list`	`object \	None`	Loaded symptoms placeholder

Runtime behavior

- Allows all origins, methods, and headers through CORS.
- Loads model and symptoms placeholders at startup.
- Sits behind the Node AI gateway instead of being called directly from the browser.

`ai-service/report_analyzer.py`

`ai-service/report_analyzer.py`

This module performs OCR and Gemini-based report analysis.

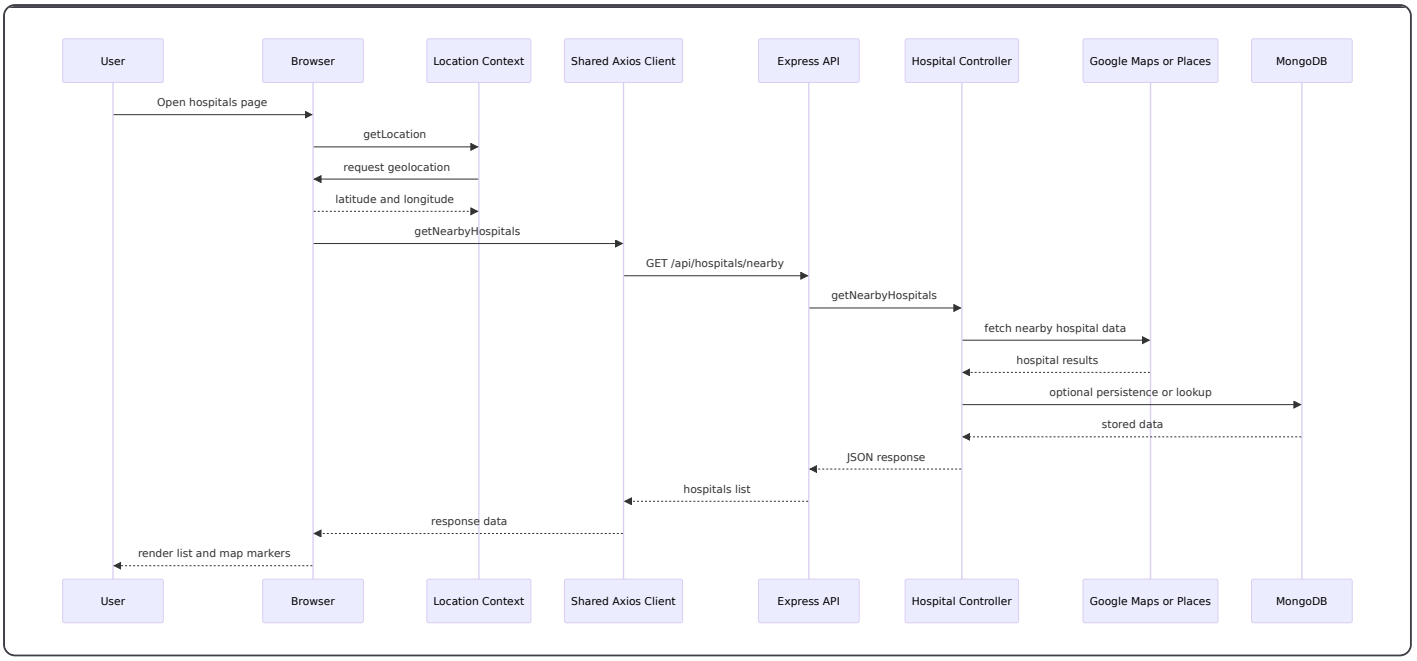
Method	Description
`analyze_medical_report`	Extracts text from an image or PDF, runs OCR, and sends the extracted content to Gemini for structured analysis

Processing stages

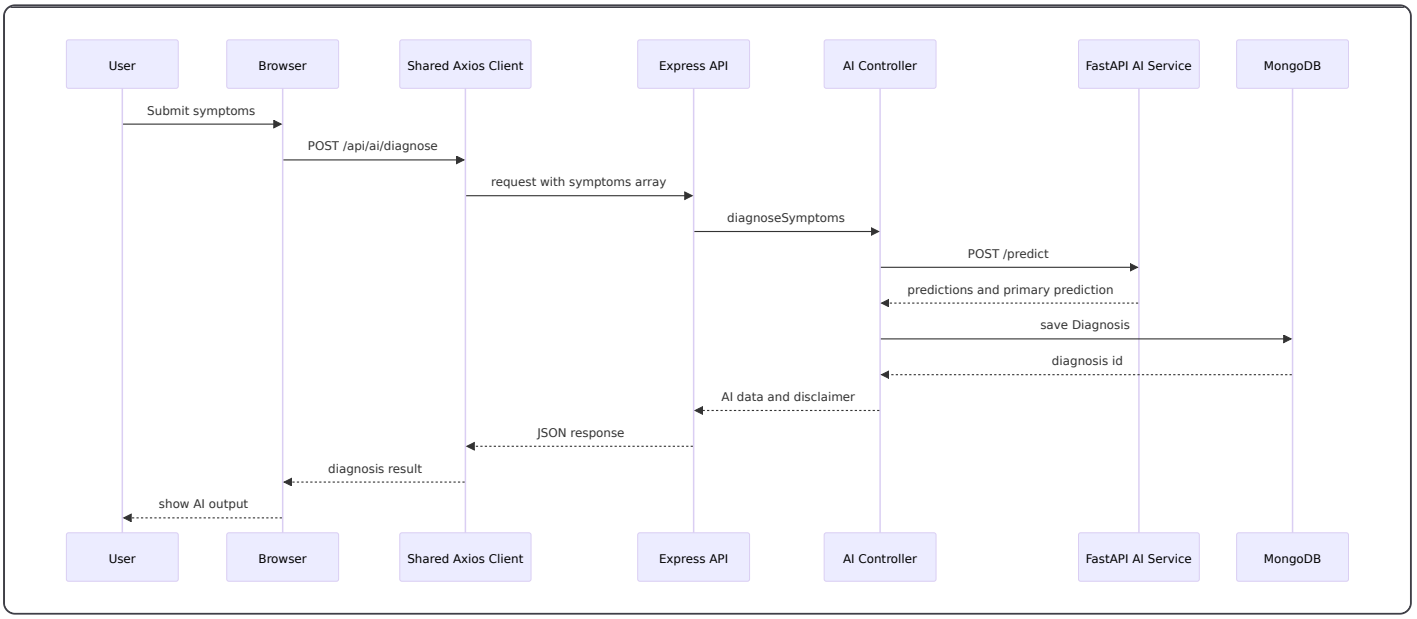
1. Loads `GEMINI_API_KEY` from `.env`.
2. Configures `pytesseract` with the Windows Tesseract path.
3. Detects file type by extension.
4. Runs OCR for `.png`, `.jpg`, `.jpeg`, or `.pdf`.
5. Returns explicit error objects for unsupported files, OCR failures, empty extraction, and missing Gemini credentials.
6. Sends a prompt asking for summary, findings, advice, precautions, medicines, and urgency.
7. Returns the model text along with a disclaimer and the first part of the extracted text.

Request Flow and Service Composition

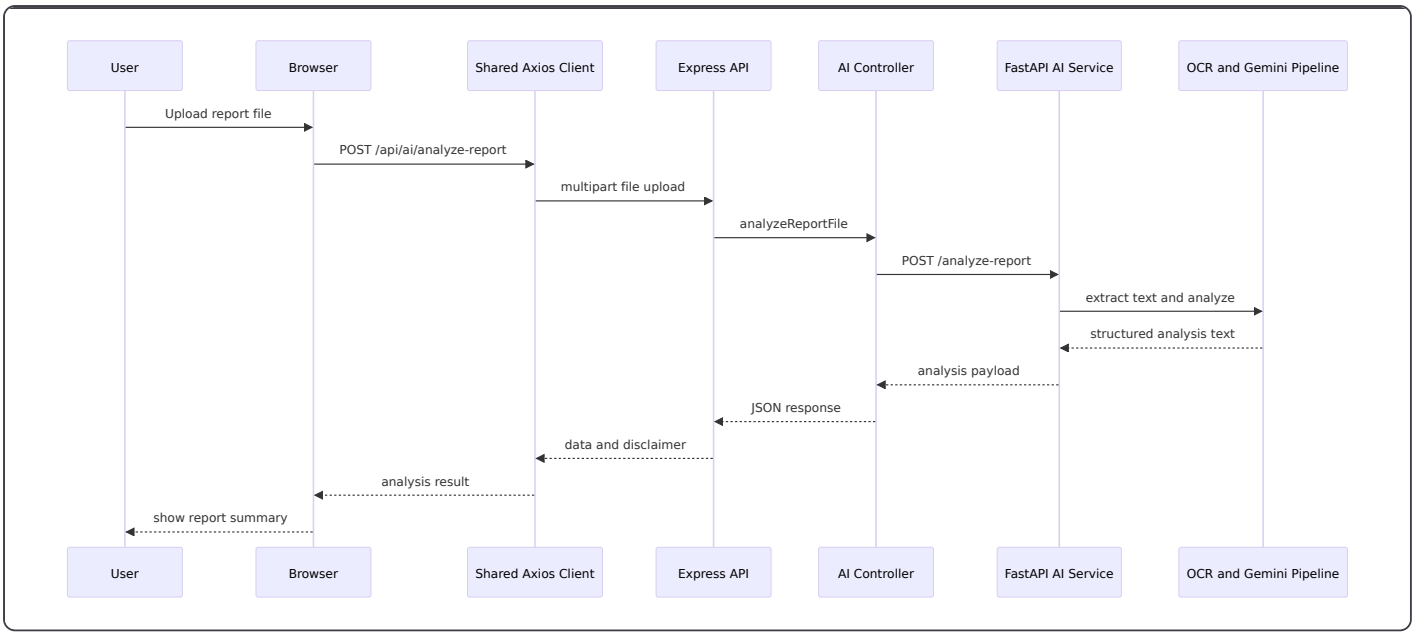
Nearby hospital discovery flow



Symptom diagnosis flow



Report analysis flow



API Integration

Health check

Get Health Status

{ } Get Health Status

Checks whether the Express runtime is running

GET `<BackendApiBaseUrl>/health`

Code examples

Copy

Curl Javascript Python

```
curl --request GET \
  --url "<BackendApiBaseUrl>/health"
```

Responses

200

Server is running

```
{
    "success": true,
    "message": "MediAccess Server is running",
    "timestamp": "2026-04-02T12:00:00.000Z"
}
```

Authentication surface

Register User

{ } Register User

Creates a standard user account and returns a JWT session

POST `<BackendApiBaseUrl>/api/auth/register`

Headers

`Content-Type` string required

application/json

Code examples

Copy

Curl Javascript Python

```
curl --request POST \
  --url "<BackendApiBaseUrl>/api/auth/register" \
  --header 'Content-Type: application/json' \
  --data '{
    "name": "Amina Khan",
    "email": "amina@example.com",
    "password": "SecurePass123",
    "phone": "+91 9876543210",
    "city": "Delhi"
}'
```

Responses

200

Account created

```
{
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
        "id": "66f1b3f3e8f1d25c9a1a0001",
        "name": "Amina Khan",
        "email": "amina@example.com",
        "phone": "+91 9876543210",
        "city": "Delhi",
        "role": "user"
    }
}
```

Register Hospital

{ } Register Hospital

Creates a hospital administrator account and hospital profile

POST `<BackendApiBaseUrl>/api/auth/register-hospital`

Headers

Content-Type string required

application/json

Code examples

Copy

Curl Javascript Python

```
curl --request POST \
  --url "<BackendApiBaseUrl>/api/auth/register-hospital" \
  --header 'Content-Type: application/json' \
  --data '{
    "name": "Dr. Maya Singh",
    "email": "admin@cityhospital.example",
    "password": "SecurePass123",
    "phone": "+91 9876501234",
    "hospitalName": "City General Hospital",
    "hospitalAddress": "123 Healthcare Ave, New Delhi",
    "hospitalPhone": "+91 1144001122",
    "registrationNumber": "REG-2024-2048",
    "hospitalDescription": "Multi-specialty hospital with emergency care",
    "hospitalSpecialties": [
        "Cardiology",
        "Radiology",
        "Emergency Medicine"
    ],
    "totalBeds": 200,
    "availableBeds": 80,
    "hospitalLat": 28.6139,
    "hospitalLng": 77.209
}'
```

Responses

200

Hospital account created


```
{
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
        "id": "66f1b3f3e8f1d25c9a1a0002",
        "name": "Dr. Maya Singh",
        "email": "admin@cityhospital.example",
        "phone": "+91 9876501234",
        "role": "hospital_admin",
        "hospitalName": "City General Hospital",
        "hospitalAddress": "123 Healthcare Ave, New Delhi",
        "hospitalPhone": "+91 1144001122",
        "hospitalDescription": "Multi-specialty hospital with emergency care",
        "hospitalSpecialties": [
            "Cardiology",
            "Radiology",
            "Emergency Medicine"
        ],
        "totalBeds": 200,
        "availableBeds": 80,
        "hospitalLat": 28.6139,
        "hospitalLng": 77.209,
        "isVerified": false
    }
}
```


Login

{ } Login

Authenticates a user and returns a JWT session

POST `<BackendApiBaseUrl>/api/auth/login`

Headers

`Content-Type` string required

application/json

Code examples

Copy

Curl Javascript Python

```
curl --request POST \
  --url "<BackendApiBaseUrl>/api/auth/login" \
  --header 'Content-Type: application/json' \
  --data '{
    "email": "amina@example.com",
    "password": "SecurePass123"
}'
```

Responses

200

Login successful

```
{
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
        "id": "66f1b3f3e8f1d25c9a1a0001",
        "name": "Amina Khan",
        "email": "amina@example.com",
        "role": "user"
    }
}
```

Get Profile

{ } Get Profile

Returns the authenticated user profile

GET `<BackendApiBaseUrl>/api/auth/profile`

Headers

`Authorization` string required

Bearer <token>

Code examples

Copy

Curl Javascript Python

```
curl --request GET \
  --url "<BackendApiBaseUrl>/api/auth/profile" \
  --header 'Authorization: Bearer <token>'
```

Responses

200

Profile returned

```
{
    "id": "66f1b3f3e8f1d25c9a1a0001",
    "name": "Amina Khan",
    "email": "amina@example.com",
    "phone": "+91 9876543210",
    "city": "Delhi",
    "role": "user"
}
```

Update Profile

{ } Update Profile

Updates the authenticated user profile

PUT `<BackendApiBaseUrl>/api/auth/profile`

Headers

Authorization string required

Bearer <token>

Content-Type string required

application/json

Code examples

Copy

Curl Javascript Python

```
curl --request PUT \
  --url "<BackendApiBaseUrl>/api/auth/profile" \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "name": "Amina Khan",
    "phone": "+91 9876543210",
    "city": "Delhi"
}'
```

Responses

200

Profile updated

```
{
    "id": "66f1b3f3e8f1d25c9a1a0001",
    "name": "Amina Khan",
    "email": "amina@example.com",
    "phone": "+91 9876543210",
    "city": "Delhi",
    "role": "user"
}
```

Get Registered Hospitals

{ } Get Registered Hospitals

Returns the list of registered hospitals

GET `<BackendApiBaseUrl>/api/auth/hospitals`

Code examples

Copy

Curl Javascript Python

```
curl --request GET \
  --url "<BackendApiBaseUrl>/api/auth/hospitals"
```

Responses

200

Hospital list

```
{
    "hospitals": [
        {
            "id": "66f1b3f3e8f1d25c9a1a0002",
            "name": "City General Hospital",
            "address": "123 Healthcare Ave, New Delhi",
            "phone": "+91 1144001122",
            "rating": 4.5
        }
    ]
}
```

Nearby facility discovery

Get Nearby Hospitals

{ } Get Nearby Hospitals

Fetches hospitals near the current user location

GET `<BackendApiBaseUrl>/api/hospitals/nearby`

Query parameters

latitude string required

28.6139

longitude string required

77.2090

radius string

5000

Code examples

Copy

Curl Javascript Python

```
curl --request GET \
  --url "<BackendApiBaseUrl>/api/hospitals/nearby?latitude=28.6139&longitude=77.2090&radius=5000"
```

Responses

200

Nearby hospitals


```
{
    "success": true,
    "hospitals": [
        {
            "id": "place_1",
            "name": "City General Hospital",
            "lat": 28.6139,
            "lng": 77.209,
            "address": "123 Healthcare Ave, New Delhi",
            "phone": "+91 1144001122",
            "rating": 4.5,
            "openNow": true
        }
    ],
    "count": 1
}
```

Get Hospital Details

{ } Get Hospital Details

Fetches detailed information for a single hospital place ID

GET `<BackendApiBaseUrl>/api/hospitals/details/<placeId>`

Path parameters

placeId string required

place_1

Code examples

Copy

Curl Javascript Python

```
curl --request GET \
  --url "<BackendApiBaseUrl>/api/hospitals/details/<placeId>"
```

Responses

200

Hospital details

```
{
    "success": true,
    "hospital": {
        "id": "place_1",
        "name": "City General Hospital",
        "lat": 28.6139,
        "lng": 77.209,
        "address": "123 Healthcare Ave, New Delhi",
        "phone": "+91 1144001122",
        "rating": 4.5,
        "openNow": true
    }
}
```

AI service bridge

Diagnose Symptoms

{ } Diagnose Symptoms

Sends symptom arrays to the optional Python AI runtime and stores the returned diagnosis

POST `<BackendApiBaseUrl>/api/ai/diagnose`

Headers

`Authorization` string required

Bearer <token>

`Content-Type` string required

application/json

Code examples

Copy

Curl Javascript Python

```
curl --request POST \
  --url "<BackendApiBaseUrl>/api/ai/diagnose" \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "symptoms": [
        "fever",
        "dry cough",
        "fatigue"
    ]
}'
```

Responses

200

Diagnosis returned

```
{
    "success": true,
    "data": {
        "predictions": [
            "Influenza",
            "Common Cold",
            "Allergy"
        ],
        "primary_prediction": "Influenza",
        "disclaimer": "This is not a medical diagnosis. Consult a licensed doctor before taking any
medication."
    },
    "diagnosisId": "66f1b3f3e8f1d25c9a1a1001",
    "disclaimer": "This is not a medical diagnosis. Consult a licensed doctor before taking any
medication."
}
```

Analyze Report File

{ } Analyze Report File

Uploads a report file to the Python OCR and Gemini service

POST `<BackendApiBaseUrl>/api/ai/analyze-report`

Headers

Authorization string required

Bearer <token>

Content-Type string required

multipart/form-data

Code examples

Copy

Curl Javascript Python

```
curl --request POST \
  --url "<BackendApiBaseUrl>/api/ai/analyze-report" \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: multipart/form-data'
```

Responses

200

Analysis returned

```
{
    "success": true,
    "data": {
        "analysis": "The report indicates elevated glucose and low hemoglobin.",
        "disclaimer": "This is not a medical diagnosis. Consult a licensed doctor before taking any
medication.",
        "raw_text_extracted": "Glucose 182 mg/dL Hemoglobin 9.1 g/dL..."
    },
    "disclaimer": "This is not a medical diagnosis. Consult a licensed doctor before taking any
medication."
}
```

Analyze Existing Report

{ } Analyze Existing Report

Returns a stored report record and a coming soon message instead of running OCR

POST `<BackendApiBaseUrl>/api/reports/<id>/analyze`

Headers

Authorization string required

Bearer <token>

Content-Type string required

application/json

Path parameters

`id` string required

66f1b3f3e8f1d25c9a1b0001

Code examples

Copy

Curl Javascript Python

```
curl --request POST \
  --url "<BackendApiBaseUrl>/api/reports/<id>/analyze" \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json'
```

Responses

200

Stored report returned


```
{
    "success": true,
    "message": "Feature coming soon: analyzing previously uploaded reports.",
    "data": {
        "patient": "66f1b3f3e8f1d25c9a1a0001",
        "title": "MRI Brain Scan",
        "category": "MRI",
        "fileUrl": "/uploads/reports/mri-brain-scan.pdf",
        "fileName": "mri-brain-scan.pdf",
        "dateConducted": "2026-04-02T12:00:00.000Z",
        "aiAnalysis": {
            "summary": "Normal study",
            "findings": [
                "No acute abnormality"
            ],
            "recommendations": [
                "Follow up with neurologist"
            ],
            "confidenceScore": 0.92,
            "analyzedAt": "2026-04-02T12:00:00.000Z"
        },
        "createdAt": "2026-04-02T12:00:00.000Z",
        "updatedAt": "2026-04-02T12:00:00.000Z"
    }
}
```

Blood request workflow

Create Blood Request

{ } Create Blood Request

Creates a blood request and stores the caller as the requester

POST `<BackendApiBaseUrl>/api/blood-requests`

Headers

Authorization string required

Bearer <token>

Content-Type string required

application/json

Code examples

Copy

Curl Javascript Python

```
curl --request POST \
  --url "<BackendApiBaseUrl>/api/blood-requests" \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "bloodGroup": "O+",
    "hospital": "City General Hospital",
    "city": "Delhi",
    "urgency": "high",
    "contact": "+91 9876543210",
    "unitsNeeded": 2,
    "notes": "Patient needs urgent transfusion",
    "latitude": 28.6139,
    "longitude": 77.209
}'
```

Responses

201

Blood request created

```
{
    "success": true,
    "message": "Blood request created successfully",
    "request": {
        "id": "66f1b3f3e8f1d25c9a1c0001",
        "bloodGroup": "O+",
        "hospital": "City General Hospital",
        "urgency": "high",
        "status": "pending"
    }
}
```

Get Blood Requests

{ } Get Blood Requests

Lists pending blood requests with optional filters

GET `<BackendApiBaseUrl>/api/blood-requests`

Query parameters

bloodGroup string

O+

city string

Delhi

urgency string

high

latitude string

28.6139

longitude string

77.2090

radius string

10

Code examples

Copy

Curl Javascript Python

```
curl --request GET \
  --url "<BackendApiBaseUrl>/api/blood-requests?
bloodGroup=O+&city=Delhi&urgency=high&latitude=28.6139&longitude=77.2090&radius=10"
```

Responses

200

Blood requests returned

```
{
    "success": true,
    "requests": [
        {
            "id": "66f1b3f3e8f1d25c9a1c0001",
            "bloodGroup": "O+",
            "hospital": "City General Hospital",
            "city": "Delhi",
            "urgency": "high",
            "contact": "+91 9876543210",
            "unitsNeeded": 2,
            "status": "pending",
            "createdAt": "2026-04-02T12:00:00.000Z",
            "requesterName": "Amina Khan",
            "requesterEmail": "amina@example.com"
        }
    ],
    "count": 1
}
```


Get My Blood Requests

{ } Get My Blood Requests

Returns the current caller's blood requests

GET `<BackendApiBaseUrl>/api/blood-requests/my-requests`

Headers

Authorization string required

Bearer <token>

Code examples

Copy

Curl Javascript Python

```
curl --request GET \
  --url "<BackendApiBaseUrl>/api/blood-requests/my-requests" \
  --header 'Authorization: Bearer <token>'
```

Responses

200

My blood requests

```
{
    "success": true,
    "requests": [
        {
            "id": "66f1b3f3e8f1d25c9a1c0001",
            "bloodGroup": "O+",
            "hospital": "City General Hospital",
            "city": "Delhi",
            "urgency": "high",
            "contact": "+91 9876543210",
            "unitsNeeded": 2,
            "status": "pending",
            "notes": "Patient needs urgent transfusion",
            "createdAt": "2026-04-02T12:00:00.000Z"
        }
    ],
    "count": 1
}
```

Update Blood Request

{ } Update Blood Request

Updates status and notes for a blood request

PUT `<BackendApiBaseUrl>/api/blood-requests/<id>`

Headers

Authorization string required

Bearer <token>

Content-Type string required

application/json

Path parameters

id string required

66f1b3f3e8f1d25c9a1c0001

Code examples

Copy

Curl Javascript Python

```
curl --request PUT \
  --url "<BackendApiBaseUrl>/api/blood-requests/<id>" \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "status": "fulfilled",
    "notes": "Units arranged through donor network"
}'
```

Responses

200

Blood request updated

```
{
    "success": true,
    "message": "Blood request updated successfully",
    "request": {
        "_id": "66f1b3f3e8f1d25c9a1c0001",
        "status": "fulfilled",
        "notes": "Units arranged through donor network"
    }
}
```

Delete Blood Request

{ } Delete Blood Request

Deletes a blood request by ID

DELETE `<BackendApiBaseUrl>/api/blood-requests/<id>`

Headers

`Authorization` string required

Bearer <token>

Path parameters

`id` string required

66f1b3f3e8f1d25c9a1c0001

Code examples

Copy

Curl Javascript Python

```
curl --request DELETE \
  --url "<BackendApiBaseUrl>/api/blood-requests/<id>" \
  --header 'Authorization: Bearer <token>'
```

Responses

200

Blood request deleted

```
{
    "success": true,
    "message": "Blood request deleted successfully"
}
```

Resource reporting workflow

Report Resource

{ } Report Resource

Submits a medical resource report

POST `<BackendApiBaseUrl>/api/resources`

Headers

Authorization string required

Bearer <token>

Content-Type string required

application/json

Code examples

Copy

Curl Javascript Python

```
curl --request POST \
  --url "<BackendApiBaseUrl>/api/resources" \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "type": "oxygen",
    "address": "42 Relief Street, Delhi",
    "city": "Delhi",
    "quantity": "10 cylinders",
    "contact": "+91 9876509876",
    "notes": "Available after 6 PM",
    "latitude": 28.6139,
    "longitude": 77.209
}'
```

Responses

201

Resource reported

```
{
    "success": true,
    "message": "Resource reported successfully",
    "report": {
        "id": "66f1b3f3e8f1d25c9a1d0001",
        "type": "oxygen",
        "address": "42 Relief Street, Delhi",
        "status": "pending"
    }
}
```

Get Verified Resources

{ } Get Verified Resources

Lists verified resource reports with optional filters

GET `<BackendApiBaseUrl>/api/resources/verified`

Query parameters

type string

oxygen

city string

Delhi

latitude string

28.6139

longitude string

77.2090

radius string

10

Code examples

Copy

Curl Javascript Python

```
curl --request GET \
  --url "<BackendApiBaseUrl>/api/resources/verified?
type=oxygen&city=Delhi&latitude=28.6139&longitude=77.2090&radius=10"
```

Responses

200

Verified resources

```
{
    "success": true,
    "resources": [
        {
            "id": "66f1b3f3e8f1d25c9a1d0001",
            "type": "oxygen",
            "address": "42 Relief Street, Delhi",
            "city": "Delhi",
            "quantity": "10 cylinders",
            "contact": "+91 9876509876",
            "notes": "Available after 6 PM",
            "reporterName": "Amina Khan",
            "reporterPhone": "+91 9876543210",
            "createdAt": "2026-04-02T12:00:00.000Z"
        }
    ],
    "count": 1
}
```

Get Pending Resources

{ } Get Pending Resources

Lists pending resource reports for admin review

GET `<BackendApiBaseUrl>/api/resources/pending`

Headers

Authorization string required

Bearer <token>

Code examples

Copy

Curl Javascript Python

```
curl --request GET \
  --url "<BackendApiBaseUrl>/api/resources/pending" \
  --header 'Authorization: Bearer <token>'
```

Responses

200

Pending resources

```
{
    "success": true,
    "resources": [
        {
            "_id": "66f1b3f3e8f1d25c9a1d0001",
            "type": "oxygen",
            "address": "42 Relief Street, Delhi",
            "city": "Delhi",
            "status": "pending"
        }
    ],
    "count": 1
}
```

Get My Resource Reports

{ } Get My Resource Reports

Returns the caller's submitted resource reports

GET `<BackendApiBaseUrl>/api/resources/my-reports`

Headers

Authorization string required

Bearer <token>

Code examples

Copy

Curl Javascript Python

```
curl --request GET \
  --url "<BackendApiBaseUrl>/api/resources/my-reports" \
  --header 'Authorization: Bearer <token>'
```

Responses

200

My resource reports

```
{
    "success": true,
    "reports": [
        {
            "_id": "66f1b3f3e8f1d25c9a1d0001",
            "type": "oxygen",
            "address": "42 Relief Street, Delhi",
            "city": "Delhi",
            "status": "pending"
        }
    ],
    "count": 1
}
```

Verify Resource

{ } Verify Resource

Marks a resource as verified and stores the verifier

PUT `<BackendApiBaseUrl>/api/resources/<id>/verify`

Headers

Authorization string required

Bearer <token>

Content-Type string required

application/json

Path parameters

id string required

66f1b3f3e8f1d25c9a1d0001

Code examples

Copy

Curl Javascript Python

```
curl --request PUT \
  --url "<BackendApiBaseUrl>/api/resources/<id>/verify" \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "status": "verified"
}'
```

Responses

200

Resource verified

```
{
    "success": true,
    "message": "Resource verified successfully",
    "resource": {
        "_id": "66f1b3f3e8f1d25c9a1d0001",
        "type": "oxygen",
        "address": "42 Relief Street, Delhi",
        "status": "verified",
        "verified": true,
        "verifiedBy": "66f1b3f3e8f1d25c9a1a0002"
    }
}
```

Reject Resource

{ } Reject Resource

Marks a resource as rejected

PUT `<BackendApiBaseUrl>/api/resources/<id>/reject`

Headers

Authorization string required

Bearer <token>

Content-Type string required

application/json

Path parameters

id string required

66f1b3f3e8f1d25c9a1d0001

Code examples

Copy

Curl Javascript Python

```
curl --request PUT \
  --url "<BackendApiBaseUrl>/api/resources/<id>/reject" \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "status": "rejected"
}'
```

Responses

200

Resource rejected

```
{
    "success": true,
    "message": "Resource rejected",
    "resource": {
        "_id": "66f1b3f3e8f1d25c9a1d0001",
        "type": "oxygen",
        "address": "42 Relief Street, Delhi",
        "status": "rejected"
    }
}
```

Delete Resource

{ } Delete Resource

Deletes a resource report by ID

DELETE `<BackendApiBaseUrl>/api/resources/<id>`

Headers

`Authorization` string required

Bearer <token>

Path parameters

`id` string required

66f1b3f3e8f1d25c9a1d0001

Code examples

Copy

Curl Javascript Python

```
curl --request DELETE \
  --url "<BackendApiBaseUrl>/api/resources/<id>" \
  --header 'Authorization: Bearer <token>'
```

Responses

200

Resource deleted

```
{
    "success": true,
    "message": "Resource deleted successfully"
}
```

Admin surface

Get Dashboard Stats

{ } Get Dashboard Stats

Fetches hospital admin dashboard metrics

GET `<BackendApiBaseUrl>/api/admin/dashboard`

Headers

`Authorization` string required

Bearer <token>

Code examples

Copy

Curl Javascript Python

```
curl --request GET \
  --url "<BackendApiBaseUrl>/api/admin/dashboard" \
  --header 'Authorization: Bearer <token>'
```

Responses

200

Dashboard stats

```
{
    "data": {
        "total": 24,
        "confirmed": 11,
        "pending": 8,
        "cancelled": 3,
        "todayCount": 5,
        "availableBeds": 80,
        "totalBeds": 200,
        "rating": 4.5
    }
}
```

Get Hospital Appointments

{ } Get Hospital Appointments

Lists appointments for a hospital admin, optionally filtered by status

GET `<BackendApiBaseUrl>/api/admin/appointments`

Headers

`Authorization` string required

Bearer <token>

Query parameters

`status` string

Confirmed

Code examples

Copy

Curl Javascript Python

```
curl --request GET \
  --url "<BackendApiBaseUrl>/api/admin/appointments?status=Confirmed" \
  --header 'Authorization: Bearer <token>'
```

Responses

200

Appointments returned


```
{
    "data": [
        {
            "_id": "66f1b3f3e8f1d25c9a1e0001",
            "patient": {
                "_id": "66f1b3f3e8f1d25c9a1a0001",
                "name": "Amina Khan",
                "email": "amina@example.com",
                "phone": "+91 9876543210"
            },
            "appointmentDate": "2026-04-02T10:00:00.000Z",
            "status": "Scheduled",
            "hospitalNotes": ""
        }
    ]
}
```


Update Appointment Status

{ } Update Appointment Status

Updates an appointment status and hospital notes

PUT `<BackendApiBaseUrl>/api/admin/appointments/<id>`

Headers

`Authorization` string required

Bearer <token>

`Content-Type` string required

application/json

Path parameters

`id` string required

66f1b3f3e8f1d25c9a1e0001

Code examples

Copy

Curl Javascript Python

```
curl --request PUT \
  --url "<BackendApiBaseUrl>/api/admin/appointments/<id>" \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "status": "Confirmed",
    "hospitalNotes": "Patient confirmed for morning slot"
}'
```

Responses

200

Appointment updated

```
{
    "data": {
        "_id": "66f1b3f3e8f1d25c9a1e0001",
        "status": "Confirmed",
        "hospitalNotes": "Patient confirmed for morning slot"
    }
}
```

Update Hospital Profile

{ } Update Hospital Profile

Updates hospital profile settings and capacity fields

PUT `<BackendApiBaseUrl>/api/admin/profile`

Headers

`Authorization` string required

Bearer <token>

`Content-Type` string required

application/json

Code examples

Copy

Curl Javascript Python

```
curl --request PUT \
  --url "<BackendApiBaseUrl>/api/admin/profile" \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "hospitalName": "City General Hospital",
    "hospitalAddress": "123 Healthcare Ave, New Delhi",
    "hospitalPhone": "+91 1144001122",
    "hospitalDescription": "Multi-specialty hospital with emergency care",
    "hospitalSpecialties": [
        "Cardiology",
        "Radiology",
        "Emergency Medicine"
    ],
    "totalBeds": 200,
    "availableBeds": 80,
    "hospitalLat": 28.6139,
    "hospitalLng": 77.209
}'
```

Responses

200

Profile updated

```
{
    "data": {
        "hospitalName": "City General Hospital",
        "hospitalAddress": "123 Healthcare Ave, New Delhi",
        "hospitalPhone": "+91 1144001122",
        "hospitalDescription": "Multi-specialty hospital with emergency care",
        "hospitalSpecialties": [
            "Cardiology",
            "Radiology",
            "Emergency Medicine"
        ],
        "totalBeds": 200,
        "availableBeds": 80,
        "hospitalLat": 28.6139,
        "hospitalLng": 77.209
    }
}
```

Get Hospital Patients

{ } Get Hospital Patients

Lists patient records visible to the hospital admin

GET `<BackendApiBaseUrl>/api/admin/patients`

Headers

`Authorization` string required

Bearer <token>

Code examples

Copy

Curl Javascript Python

```
curl --request GET \
  --url "<BackendApiBaseUrl>/api/admin/patients" \
  --header 'Authorization: Bearer <token>'
```

Responses

200

Patient records


```
{
    "data": [
        {
            "_id": "66f1b3f3e8f1d25c9a1a0001",
            "name": "Amina Khan",
            "email": "amina@example.com",
            "phone": "+91 9876543210",
            "city": "Delhi",
            "bloodGroup": "O+",
            "lastAppointment": "2026-04-02T10:00:00.000Z"
        }
    ]
}
```


Integration Points

- `frontend/index.html` preloads the Google Maps JS API script, which supports location-aware discovery experiences in the browser.
- `frontend/src/services/api.js` is the only HTTP client used by the SPA facades.
- `backend/server.js` serves uploaded files from `/uploads`, which supports report workflows.
- `backend/controllers/aiController.js` is the runtime bridge between the Express API and the optional FastAPI service.
- `backend/config/db.js` is the persistence boundary for all Mongoose-backed workflows.
- `ai-service/report_analyzer.py` provides OCR and Gemini analysis for uploaded medical reports.

- `AuthContext` and `LocationContext` provide shared user/session and geolocation state across the SPA.
- `HospitalAdminDashboard`, `AdminAppointments`, `AdminPatients`, and `AdminSettings` compose the hospital management surface over the shared admin API.
- `BloodRequests`, `BloodDonors`, `ReportResource`, and `ResourceForm` extend the platform from locator workflows into community support workflows.
- `AIAssistant` and `InteractiveAnatomy` extend the platform into clinical education and assisted interpretation.

Dependencies

Runtime	Packages and Services	Role
Backend	`express`, `cors`, `dotenv`, `mongoose`, `jsonwebtoken`, `bcryptjs`, `multer`, `axios`, `express-validator`	API runtime, auth, data persistence, file uploads, AI bridge, request validation
Frontend	`react`, `react-dom`, `react-router-dom`, `axios`, `react-toastify`, `framer-motion`, `three`, `vanta`, `@googlemaps/js-api-loader`	SPA shell, routing, async transport, notifications, animation, visual effects, maps support
AI service	`FastAPI`, `CORSMiddleware`, `pytesseract`, `pdf2image`, `google.generativeai`, `Pillow`, `joblib`, `pandas`, `numpy`	OCR pipeline, PDF/image extraction, Gemini integration, model loading
External systems	MongoDB, Google Maps JS API, Gemini, Tesseract	Persistence, browser mapping, AI analysis, OCR

Error Handling

The repository uses layered error handling rather than a single global pattern.

- `backend/server.js` ends with a 404 JSON response and `errorHandler` middleware.
- `backend/controllers/aiController.js` converts AI service failures into structured 4xx/5xx JSON responses and logs upstream failure details.
- `frontend/src/services/api.js` clears session state and redirects to `/login` on a `401`.
- Feature pages use `toast.error` for user-facing request failures and `toast.success` for completed operations.
- `LocationContext` stores geolocation failures directly in the shared `location.error` field.
- `ai-service/report_analyzer.py` returns explicit error objects for unsupported files, missing OCR support, empty extraction, and missing Gemini credentials.

Caching Strategy

There is no dedicated application cache service in the repository-level architecture. The effective state reuse comes from:

- `localStorage` for auth tokens and serialized user profiles.
- `localStorage` for pharmacy session data.
- Context state in `AuthContext` and `LocationContext`.
- Browser-side request reuse through the shared Axios instance.

Testing Considerations

- Backend startup should verify MongoDB connection failure exits the process through `connectDB`.
- SPA boot should verify `AuthProvider` restores the user from `localStorage`.
- `api.js` should verify 401 responses clear auth state and redirect to `/login`.
- `LocationContext` should verify both successful geolocation and browser-denied geolocation states.
- AI gateway flows should verify:
  - symptom validation rejects non-array payloads,
  - uploaded report files are cleaned up after success and after errors,
  - missing Python runtime returns a service-unavailable response,
  - `analyzeReport` returns the stored `Report` object and its coming-soon message.
- Upload-driven flows should verify static `/uploads` serving remains aligned with report and file-path usage.
- Hospital admin flows should verify the `hospital_admin` guard in the browser shell and the server-side access restrictions remain aligned.

Key Classes Reference

Class	Responsibility
`server.js`	Boots the Express runtime, mounts routes, and exposes health, uploads, and error handling
`db.js`	Connects the Node API to MongoDB
`api.js`	Provides the shared Axios client and auth/error interceptors
`main.jsx`	Mounts the React app into the DOM
`App.jsx`	Defines SPA routing, layout visibility, and provider composition
`AuthContext.jsx`	Holds session state and login/logout/update helpers
`LocationContext.jsx`	Holds browser geolocation state
`aiController.js`	Bridges Express to the optional Python AI service
`bloodRequestController.js`	Handles blood request persistence and filtering
`resourceController.js`	Handles resource reporting and moderation
`main.py`	Boots the FastAPI AI service
`report_analyzer.py`	Performs OCR and Gemini analysis for medical reports